

Assignment 3: Bridge Design Pattern

Option D: Remote Controls

Student: Aiym Issabayeva

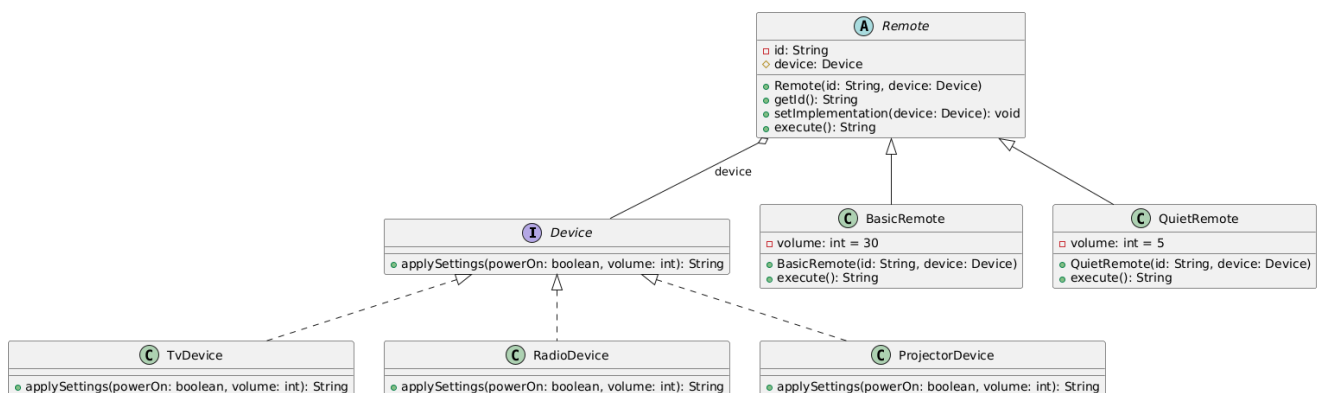
Course: Software Design Patterns

1. Introduction

In this assignment, I implemented the Bridge Design Pattern using the Remote Controls option. The project has two separate hierarchies: remote controls and devices. The main idea is to separate the abstraction from the implementation, so both sides can be changed independently.

In my project, Remote is the abstraction and Device is the implementor. BasicRemote and QuietRemote are different types of remotes, while TvDevice, RadioDevice and ProjectorDevice are different device implementations.

2. UML Diagram



The UML diagram shows two separate hierarchies. Remote has a reference to Device. This relationship connects the abstraction side with the implementation side and creates the Bridge.

3. Implementation

The Device interface contains the `applySettings()` method. Each concrete device implements this method and returns its own device type, power state and volume.

The abstract Remote class stores the device reference and an ID. It also has `setImplementation()`, which allows the implementation to be replaced at runtime. BasicRemote uses volume 30 and QuietRemote uses volume 5.

The `execute()` method in each remote calls `device.applySettings()`. Because the remote works through the `Device` interface, it does not need to know whether the current implementation is a TV, radio or projector.

4. Runtime Switching

Test T5 demonstrates runtime switching. I created one `BasicRemote` with `TvDevice`, executed it, and then changed its implementation to `RadioDevice` using `setImplementation()`. The remote object itself was not replaced.

I used the `==` operator to check that both references point to the same object. I also checked that the remote ID stays unchanged. The result changes from TV output to RADIO output because only the implementation reference changes.

5. Demo Tests

Test	What is checked
T1	BasicRemote + TvDevice
T2	BasicRemote + RadioDevice
T3	QuietRemote + TvDevice
T4	QuietRemote + RadioDevice
T5	Switch TvDevice to RadioDevice on the same BasicRemote object
T6	BasicRemote + ProjectorDevice
T7	QuietRemote + ProjectorDevice

```
ajym@192 Assignment3 % java -cp out Main --demo
T1 PASS | BasicRemote + TvDevice | result=TV | power=ON | volume=30
T2 PASS | BasicRemote + RadioDevice | result=RADIO | power=ON | volume=30
T3 PASS | QuietRemote + TvDevice | result=TV | power=ON | volume=5
T4 PASS | QuietRemote + RadioDevice | result=RADIO | power=ON | volume=5
T5 PASS | sameObject=true | stateUnchanged=true
  before=TV | power=ON | volume=30 | after=RADIO | power=ON | volume=30
T6 PASS | BasicRemote + ProjectorDevice | result=PROJECTOR | power=ON | volume=30
T7 PASS | QuietRemote + ProjectorDevice | result=PROJECTOR | power=ON | volume=5
SUMMARY: 7/7 PASS
```

The final result of the demo is SUMMARY: 7/7 PASS.

6. Clean Code

Meaningful names: Classes and methods have clear names such as BasicRemote, QuietRemote, Device and applySettings().

```
public class BasicRemote extends Remote { 9 us
```

Single Responsibility Principle: Remote classes define remote behavior, while device classes are responsible for device-specific output.

```
@Override 2 usages 2 ajym
public String applySettings(boolean powerOn, int volume) {
    return "TV | power=" + (powerOn ? "ON" : "OFF") + " | volume=" + volume;
}
```

```
@Override 8 usages 2 ajym
public String execute() {
    return device.applySettings(powerOn: true, volume);
}
```

Small methods: execute() short and have one clear purpose.

```
@Override 8 usages 2 ajym
public String execute() {
    return device.applySettings(powerOn: true, volume);
}
```

Avoiding duplication: Both remotes use the same Device interface instead of creating separate classes for every remote-device combination.

```
public interface Device { 10 usages 3 implementations 2 ajym
    String applySettings(boolean powerOn, int volume);
}
```

```
protected Device device; 4 usages
```

Encapsulation of state: The ID is stored inside Remote and accessed through getId(), while the implementation is changed through setImplementation().

```
private final String id; 2
```

```
public String getId() { 2 usages  @ajym  
    return id;  
}
```

7. Adding the Third Implementation

First, I completed the version with TvDevice and RadioDevice and tested T1-T5. Then I created the base commit. The base commit hash is 0e050ac.

After the base commit, I added ProjectorDevice as I3 and updated only Main.java to demonstrate T6 and T7. The existing Remote hierarchy, Device interface, TvDevice and RadioDevice were not changed. The extension.diff file shows these changes.

8. Bridge vs Adapter

Bridge separates abstraction and implementation so they can vary independently. Adapter is used to make incompatible interfaces work together. In my project, Bridge separates Remote and Device hierarchies.

9. Conclusion

In this assignment, I implemented the Bridge Design Pattern by separating Remote and Device into two independent hierarchies. This approach allows me to change devices without changing the remote classes. I also added a new ProjectorDevice without modifying the existing structure, which showed that the project is easy to extend.

References

1. Lecture 4 - The Structural Design Pattern: Bridge Pattern.
2. Assignment 3 - Bridge Pattern Instructions.